

Memoria Técnica: Proyecto MGCSS-Track API

1. Introducción y Objetivos

El presente documento detalla el desarrollo, evolución y puesta en producción de la API "MGCSS-Track", un sistema de gestión de incidencias y asignación de técnicos. El objetivo principal de este proyecto no ha sido únicamente la construcción del software, sino la aplicación rigurosa de prácticas de Mantenimiento y Gestión del Cambio, asegurando un ciclo de vida del software profesional, trazable y altamente mantenible.

2. Aclaraciones

Nosotros hemos realizado nuestro proyecto en Visual studio code con la extensión Live share que nos ha permitido trabajar todos juntos a la vez en el proyecto. El único inconveniente es que a la hora de hacer los commits salía que el que los realizaba era aquel que compartía el enlace independientemente de él que los hiciera. Por ello la mayoría de commits son realizados por Zohair al principio y a la mitad del proyecto y por Víctor una vez empezamos a trabajar con Docker porque es el que lo tiene instalado en su ordenador.

La otra aclaración es que el usuario Soyminero02 de github es Jaime.

3. Decisiones tomadas con respecto a la arquitectura

Para el desarrollo de la API MGCSS-Track, se ha optado por una arquitectura diseñada para maximizar la mantenibilidad, escalabilidad y la integración continua del sistema:

- **Arquitectura Limpia / Multicapa:** El sistema se ha estructurado en cuatro paquetes claramente diferenciados: Dominio, Servicios, Infraestructura y API. Esto garantiza el principio de responsabilidad única, aislando la lógica de negocio (entidades puras sin persistencia) de los controladores REST y de los adaptadores de base de datos.
- **Patrón DTO (Data Transfer Object):** Se ha implementado el uso de DTOs en la capa de presentación. Esta decisión arquitectónica evita exponer el modelo de dominio interno directamente a los clientes de la API, previniendo vulnerabilidades y permitiendo modificar la base de datos sin romper los contratos.
- **Contenedorización por Diseño (Docker):** Se ha decidido que el empaquetado final de la aplicación sea una imagen de Docker. Esto elimina el problema de dependencias locales y garantiza que el entorno de producción sea idéntico al de desarrollo y pruebas.
- **Validación de Calidad Integrada (Quality Gate):** La arquitectura del código está condicionada por un escrutinio estricto mediante un *pipeline* automatizado que conecta con SonarCloud. No se permite la liberación de ninguna versión que no

supere los estándares de métricas y cobertura dictados por el Desarrollo Guiado por Pruebas (TDD), asegurando el control total de la deuda técnica.

4. Instrucciones de instalación (probadas desde la release de Github)

El despliegue de la aplicación está completamente automatizado y se puede ejecutar siguiendo cualquiera de los dos métodos oficiales, ambos probados satisfactoriamente desde la última *Release* generada en Github.

Opción A: Instalación mediante Docker (Método Recomendado) Este método aprovecha la imagen construida y subida automáticamente a Docker Hub por nuestro *pipeline* de *Continuous Delivery*.

- **Requisito previo:** Tener Docker instalado y en ejecución en la máquina destino.
- **Paso 1:** Abrir un terminal de comandos.
- **Paso 2:** Ejecutar el siguiente comando para descargar la última versión y arrancar el servidor en el puerto 8080: `docker run -p 8080:8080 victornaruz/mgcss-api:latest`.
- **Paso 3:** La API estará disponible inmediatamente en <http://localhost:8080>.

Opción B: Instalación Autónoma (Archivo JAR desde GitHub Releases) Este método utiliza el artefacto empaquetado adjunto en la página de *Releases* del repositorio.

- **Requisito previo:** Tener instalado el entorno de ejecución de Java (JDK/JRE) versión 17.
- **Paso 1:** Navegar a la página principal del repositorio en Github.
- **Paso 2:** En la columna derecha, hacer clic en la sección **Releases** y seleccionar la última versión publicada.
- **Paso 3:** En la sección *Assets* del final de la página, descargar el archivo terminado en `.jar`.
- **Paso 4:** Abrir un terminal en la misma carpeta donde se ha descargado el archivo y ejecutar: `java -jar <nombre-del-archivo>.jar`.
- **Paso 5:** La API estará lista para recibir peticiones en el puerto 8080.

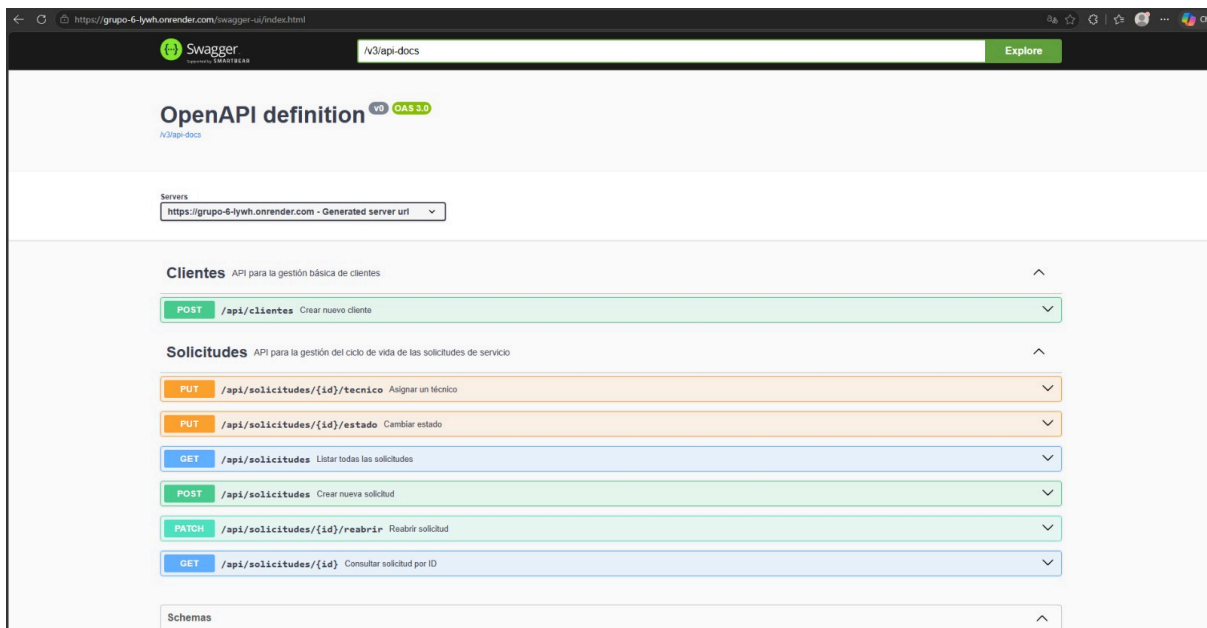
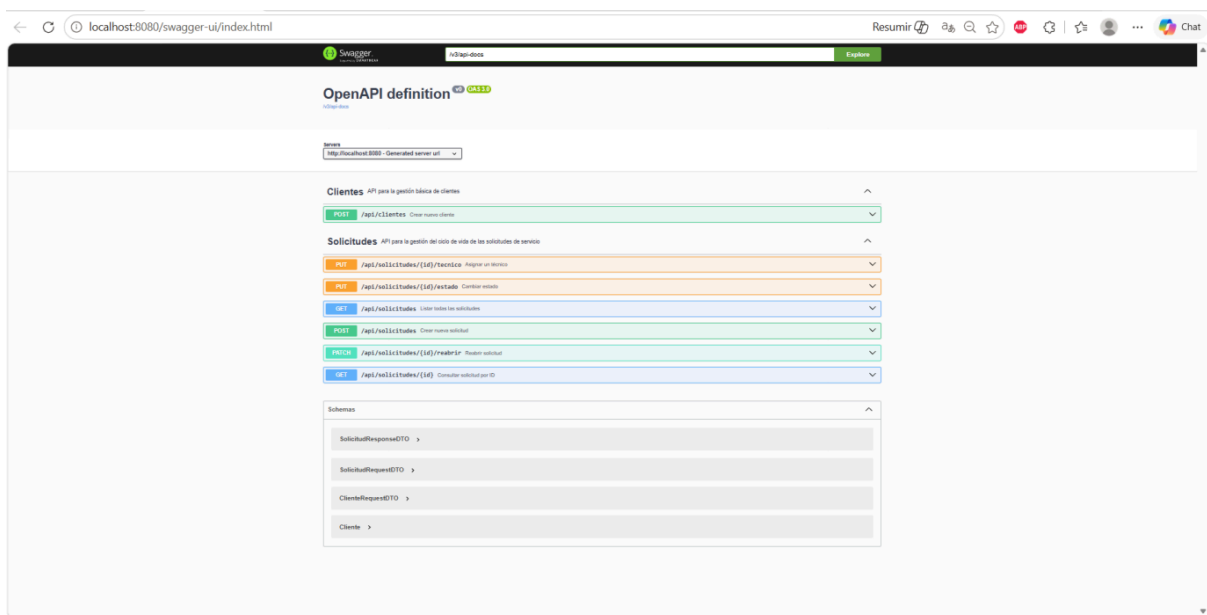
5. Extras realizados.

Se han realizado los siguientes extras:

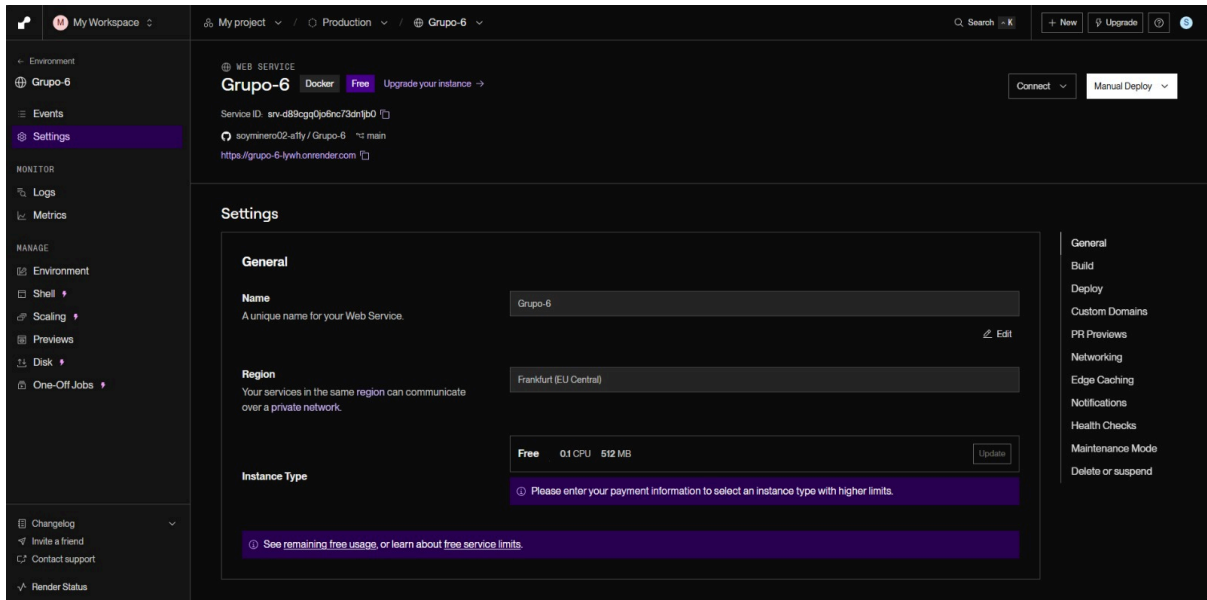
- Automatización completa de despliegue en PaaS.
- Uso de Testcontainers con PostgreSQL real.

- Dashboard de métricas.
- Métricas SLA automatizadas.
- Uso de Test-Driven Development (TDD).
- Uso de la herramienta de Gestión de Proyectos (Project) que integra Github
 - o Issues y PRs asignadas a iteraciones y a miembros del equipo.
 - Cada Entrega es una iteración.

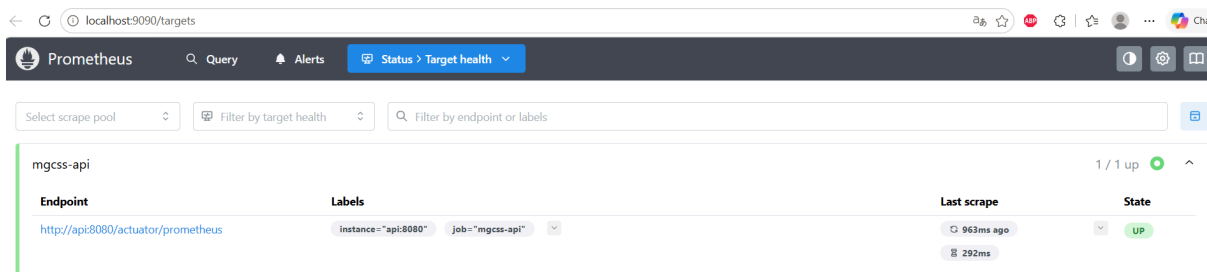
A continuación vamos a acreditar las pruebas que tenemos de que dichos puntos están hechos con las diferentes capturas que hemos hecho(a parte de que el código se encuentra en el Github):



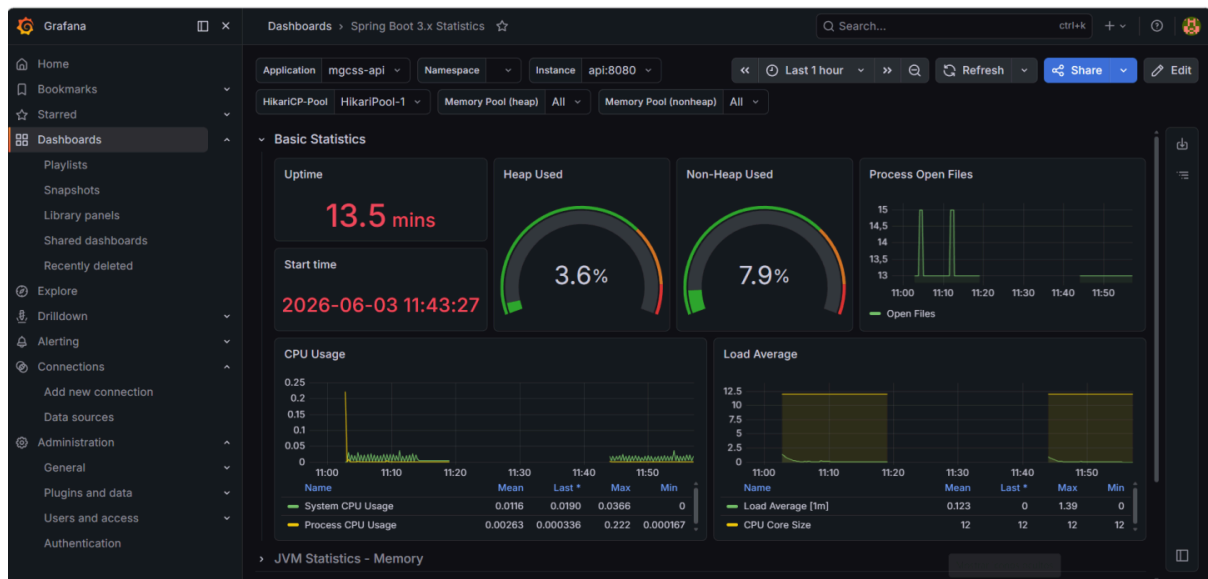
Estas 2 fotos pertenecen al swagger la primera está abierta desde el localhost utilizando la siguiente url: <http://localhost:8080/swagger-ui/index.html>. Mientras la segunda está hecha desde la siguiente url: <https://grupo-6-lywh.onrender.com/swagger-ui/index.html>



En esta foto podemos ver que el render que hemos creado está linkeado a nuestro github.



Esta foto representa nuestro prometheus que luego será utilizado para poder hacer el dashboard en grafana. Accedemos a él con la siguiente url: <http://localhost:9090/targets>



Y por último esta foto es de nuestro dashboard en grafana una vez que le hemos dado como data source nuestro prometheus. En las gráficas se puede ver que hay un parón de unos 20 minutos entre las 11:20 y las 11:40. Esto es debido a que antes de ese intervalo estuvimos trabajando en grafana y la persona encargada del docker tuvo que ir a la universidad, se paró todo y alrededor de las 11:40 volvimos a reanudarlo y posteriormente se hizo la captura. Accedemos a él con la siguiente url: <http://localhost:3000/>

La url del github es: [soyminero02-a11y/Grupo-6](https://github.com/soyminero02-a11y/Grupo-6)

La url del sonarcloud es: [Projects - soyminero02-a11y organization - SonarQube Cloud](https://sonarcloud.io/projects?organization=soyminero02-a11y)

6. Comparativa de métricas

La evolución de las métricas del proyecto se ha monitorizado de forma continua mediante la integración de SonarCloud en nuestro *pipeline* automatizado de GitHub Actions.

- **Cobertura de Pruebas (Coverage):** En las fases iniciales, la cobertura partió de un 0.0% al priorizar el diseño de la estructura arquitectónica sobre la implementación de la lógica. Tras aplicar estrictamente el Desarrollo Guiado por Pruebas (TDD) en la capa de dominio y configurar el plugin de JaCoCo, la cobertura de las nuevas implementaciones superó el mínimo del 80% exigido por el Quality Gate.
- **Fiabilidad y Seguridad:** A lo largo de todo el ciclo de desarrollo, el proyecto ha mantenido la máxima calificación ('A') en Seguridad, Fiabilidad y Mantenibilidad, garantizando el despliegue con 0 *bugs*, 0 vulnerabilidades y 0 *code smells* críticos en los cimientos del sistema.

7. Análisis de deuda técnica

El control exhaustivo de la deuda técnica ha sido un pilar central del desarrollo, abordado desde dos perspectivas fundamentales:

- **Prevención de Deuda Arquitectónica:** Se evitó el antipatrón de "Imports Tóxicos" manteniendo la capa de Dominio estructurada con objetos Java puros. Se excluyeron por completo las anotaciones de base de datos y dependencias web en esta capa, aislándola de la infraestructura para no comprometer la escalabilidad a largo plazo.
- **Bloqueo Automatizado de Defectos:** Se configuró SonarCloud para que actuara como una "aduana" automatizada. Al exigir la superación estricta del Quality Gate antes de permitir un *Merge* a la rama `main`, se bloqueó la introducción de nueva deuda técnica, rechazando físicamente cualquier Pull Request que no cumpliera con los estándares de calidad predefinidos.